

4.1. Unified Memory

This section explains the detailed behavior and use of each of the different paradigms of unified memory available. [The earlier section on unified memory](#) showed how to determine which unified memory paradigm applies and briefly introduced each.

As discussed previously there are four paradigms of unified memory programming:

- [Full support for explicit managed memory allocations](#)
- [Full support for all allocations with software coherence](#)
- [Full support for all allocations with hardware coherence](#)
- [Limited unified memory support](#)

The first three paradigms involving full unified memory support have very similar behavior and programming model and are covered in [Unified Memory on Devices with Full CUDA Unified Memory Support](#) with any differences highlighted.

The last paradigm, where unified memory support is limited, is discussed in detail in [Unified Memory on Windows, WSL, and Tegra](#).

4.1.1. Unified Memory on Devices with Full CUDA Unified Memory Support

These systems include hardware-coherent memory systems, such as NVIDIA Grace Hopper and modern Linux systems with Heterogeneous Memory Management (HMM) enabled. HMM is a software-based memory management system, providing the same programming model as hardware-coherent memory systems.

Linux HMM requires Linux kernel version 6.1.24+, 6.2.11+ or 6.3+, devices with compute capability 7.5 or higher and a CUDA driver version 535+ installed with [Open Kernel Modules](#).

Note

We refer to systems with a combined page table for both CPUs and GPUs as *hardware coherent* systems. Systems with separate page tables for CPUs and GPUs are referred to as *software-coherent*.

Hardware-coherent systems such as NVIDIA Grace Hopper offer a logically combined page table for both CPUs and GPUs, see [CPU and GPU Page Tables: Hardware Coherency vs.](#)

Software Coherency. The following section only applies to hardware-coherent systems:

- [Access Counter Migration](#)

4.1.1.1. Unified Memory: In-Depth Examples

Systems with full CUDA unified memory support, see table [Overview of Unified Memory Paradigms](#), allow the device to access any memory owned by the host process interacting with the device.

This section shows a few advanced use-cases, using a kernel that simply prints the first 8 characters of an input character array to the standard output stream:

```
__global__ void kernel(const char* type, const char* data) {  
    static const int n_char = 8;  
    printf("%s - first %d characters: '", type, n_char);  
    for (int i = 0; i < n_char; ++i) printf("%c", data[i]);  
    printf("'\\n");  
}
```

The following tabs show various ways of how this kernel may be called with system-allocated memory:

Malloc

Managed

Stack variable

File-scope static variable

Global-scope variable

Global-scope extern variable

```
void test_malloc() {  
    const char test_string[] = "Hello World";  
    char* heap_data = (char*)malloc(sizeof(test_string));  
    strncpy(heap_data, test_string, sizeof(test_string));  
    kernel<<<1, 1>>>("malloc", heap_data);  
    ASSERT(cudaDeviceSynchronize() == cudaSuccess,  
           "CUDA failed with '%s'", cudaGetErrorString(cudaGetLastError()));  
    free(heap_data);  
}
```

Note that for the extern variable, it could be declared and its memory owned and managed by a third-party library, which does not interact with CUDA at all.

Also note that stack variables as well as file-scope and global-scope variables can only be accessed through a pointer by the GPU. In this specific example, this is convenient because

the character array is already declared as a pointer: `const char*`. However, consider the following example with a global-scope integer:

```
// this variable is declared at global scope
int global_variable;

__global__ void kernel_uncompilable() {
    // this causes a compilation error: global (__host__) variables must not
    // be accessed from __device__ / __global__ code
    printf("%d\n", global_variable);
}

// On systems with pageableMemoryAccess set to 1, we can access the address
// of a global variable. The below kernel takes that address as an argument
__global__ void kernel(int* global_variable_addr) {
    printf("%d\n", *global_variable_addr);
}

int main() {
    kernel<<<1, 1>>>(&global_variable);
    ...
    return 0;
}
```

In the example above, we need to ensure to pass a *pointer* to the global variable to the kernel instead of directly accessing the global variable in the kernel. This is because global variables without the `__managed__` specifier are declared as `__host__`-only by default, thus most compilers won't allow using these variables directly in device code as of now.

4.1.1.1.1. File-backed Unified Memory

Since systems with full CUDA unified memory support allow the device to access any memory owned by the host process, they can directly access file-backed memory.

Here, we show a modified version of the initial example shown in the previous section to use file-backed memory in order to print a string from the GPU, read directly from an input file. In the following example, the memory is backed by a physical file, but the example applies to memory-backed files too.

```
__global__ void kernel(const char* type, const char* data) {
    static const int n_char = 8;
    printf("%s - first %d characters: '", type, n_char);
    for (int i = 0; i < n_char; ++i) printf("%c", data[i]);
    printf("'\\n");
}
```

```
void test_file_backed() {
    int fd = open(INPUT_FILE_NAME, O_RDONLY);
    ASSERT(fd >= 0, "Invalid file handle");
    struct stat file_stat;
    int status = fstat(fd, &file_stat);
    ASSERT(status >= 0, "Invalid file stats");
    char* mapped = (char*)mmap(0, file_stat.st_size, PROT_READ, MAP_PRIVATE, fd, 0);
    ASSERT(mapped != MAP_FAILED, "Cannot map file into memory");
    kernel<<<1, 1>>>("file-backed", mapped);
    ASSERT(cudaDeviceSynchronize() == cudaSuccess,
           "CUDA failed with '%s'", cudaGetErrorString(cudaGetLastError()));
    ASSERT(munmap(mapped, file_stat.st_size) == 0, "Cannot unmap file");
    ASSERT(close(fd) == 0, "Cannot close file");
}
```

Note that on systems without the `hostNativeAtomicSupported` property (see [Host Native Atomics](#)) including systems with Linux HMM enabled, atomic accesses to file-backed memory are not supported.

4.1.1.1.2. Inter-Process Communication (IPC) with Unified Memory

Note

As of now, using IPC with unified memory can have significant performance implications.

Many applications prefer to manage one GPU per process, but still need to use unified memory, for example for over-subscription, and access it from multiple GPUs.

CUDA IPC (see [Interprocess Communication](#)) does not support managed memory: handles to this type of memory may not be shared through any of the mechanisms discussed in this section. On systems with full CUDA unified memory support, system-allocated memory is IPC capable. Once access to system-allocated memory has been shared with other processes, the same programming model applies, similar to [File-backed Unified Memory](#).

See the following references for more information on various ways of creating IPC-capable system-allocated memory under Linux:

- [mmap with MAP_SHARED](#)
- [POSIX IPC APIs](#)
- [Linux memfd_create](#) .

Note that it is not possible to share memory between different hosts and their devices using this technique.

4.1.1.2. Performance Tuning

In order to achieve good performance with unified memory, it is important to:

- understand how paging works on your system, and how to avoid unnecessary page faults
- understand the various mechanisms allowing you to keep data local to the accessing processor
- consider tuning your application for the granularity of memory transfers of your system.

As general advice, performance hints (see [Performance Hints](#)) might provide improved performance, but using them incorrectly might degrade performance compared to the default behavior. Also note that any hint has a performance cost associated with it on the host, thus useful hints must at the very least improve performance enough to overcome this cost.

4.1.1.2.1. Memory Paging and Page Sizes

To better understand the performance implication of unified memory, it is important to understand virtual addressing, memory pages and page sizes. This sub-section attempts to define all necessary terms and explain why paging matters for performance.

All currently supported systems for unified memory use a virtual address space: this means that memory addresses used by an application represent a *virtual* location which might be *mapped* to a physical location where the memory actually resides.

All currently supported processors, including both CPUs and GPUs, additionally use memory *paging*. Because all systems use a virtual address space, there are two types of memory pages:

- Virtual pages: This represents a fixed-size contiguous chunk of virtual memory per process tracked by the operating system, which can be *mapped* into physical memory. Note that the virtual page is linked to the *mapping*: for example, a single virtual address might be mapped into physical memory using different page sizes.
- Physical pages: This represents a fixed-size contiguous chunk of memory the processor's main Memory Management Unit (MMU) supports and into which a virtual

page can be mapped.

Currently, all x86_64 CPUs use a default physical page size of 4KiB. Arm CPUs support multiple physical page sizes - 4KiB, 16KiB, 32KiB and 64KiB - depending on the exact CPU. Finally, NVIDIA GPUs support multiple physical page sizes, but prefer 2MiB physical pages or larger. Note that these sizes are subject to change in future hardware.

The default page size of virtual pages usually corresponds to the physical page size, but an application may use different page sizes as long as they are supported by the operating system and the hardware. Typically, supported virtual page sizes must be powers of 2 and multiples of the physical page size.

The logical entity tracking the mapping of virtual pages into physical pages will be referred to as a *page table*, and each mapping of a given virtual page with a given virtual size to physical pages is called a *Page Table Entry (PTE)*. All supported processors provide specific caches for the page table to speed up the translation of virtual addresses to physical addresses. These caches are called *Translation Lookaside Buffers (TLBs)*.

There are two important aspects for performance tuning of applications:

- the choice of virtual page size,
- whether the system offers a combined page table used by both CPUs and GPUs, or separate page tables for each CPU and GPU individually.

4.1.1.2.1.1. Choosing the Right Page Size

In general, small page sizes lead to less (virtual) memory fragmentation but more TLB misses, whereas larger page sizes lead to more memory fragmentation but less TLB misses. Additionally, memory migration is generally more expensive with larger page sizes compared to smaller page sizes, because we typically migrate full memory pages. This can cause larger latency spikes in an application using large page sizes. See also the next section for more details on page faults.

One important aspect for performance tuning is that TLB misses are generally significantly more expensive on the GPU compared to the CPU. This means that if a GPU thread frequently accesses random locations of unified memory mapped using a small enough page size, it might be significantly slower compared to the same accesses to unified memory mapped using a large enough page size. While a similar effect might occur for a CPU thread randomly accessing a large area of memory mapped using a small page size, the slowdown is less pronounced, meaning that the application might want to trade-off this slowdown with having less memory fragmentation.

Note that in general, applications should not tune their performance to the physical page size of a given processor, since physical page sizes are subject to change depending on the hardware. The advice above only applies to virtual page sizes.

4.1.1.2.1.2. CPU and GPU Page Tables: Hardware Coherency vs. Software Coherency

Hardware-coherent systems such as NVIDIA Grace Hopper offer a logically combined page table for both CPUs and GPUs. This is important because in order to access system-allocated memory from the GPU, the GPU uses whichever page table entry was created by the CPU for the requested memory. If that page table entry uses the default CPU page size of 4KiB or 64KiB, accesses to large virtual memory areas will cause significant TLB misses, thus significant slowdowns.

On the other hand, on software-coherent systems where the CPUs and GPUs each have their own logical page table, different performance tuning aspects should be considered: in order to guarantee coherency, these systems usually use *page faults* in case a processor accesses a memory address mapped into the physical memory of a different processor. Such a page fault means that:

- It needs to be ensured that the currently owning processor (where the physical page currently resides) cannot access this page anymore, either by deleting the page table entry or updating it.
- It needs to be ensured that the processor requesting access can access this page, either by creating a new page table entry or updating an existing entry, such that it becomes valid/active.
- The physical page backing this virtual page must be moved/migrated to the processor requesting access: this can be an expensive operation, and the amount of work is proportional to the page size.

Overall, hardware-coherent systems provide significant performance benefits compared to software-coherent systems in cases where frequent concurrent accesses to the same memory page are made by both CPU and GPU threads:

- less page-faults: these systems do not need to use page-faults for emulating coherency or migrating memory,
- less contention: these systems are coherent at cache-line granularity instead of page-size granularity, that is, when there is contention from multiple processors within a cache line, only the cache line is exchanged which is much smaller than the smallest

page-size, and when the different processors access different cache-lines within a page, then there is no contention.

This impacts the performance of the following scenarios:

- atomic updates to the same address concurrently from both CPUs and GPUs
- signaling a GPU thread from a CPU thread or vice-versa.

4.1.1.2.1.3. Mixing Hardware and Software Coherency

Some systems with hardware coherence such as NVIDIA DGX Station also support installing discrete, non-coherent GPU hardware. Accesses from a hardware coherent GPU will continue to use the hardware based coherency as described in [CPU and GPU Page Tables: Hardware Coherency vs. Software Coherency](#) while accesses from the discrete GPU will use software based coherency.

Sharing of the unified address space is permitted between both classes of GPU. This results in different performance and migration behaviour based on accessing GPU. In particular accesses from a software coherent GPU will have more page faults and memory migration, while accesses from hardware coherent GPU will fault less and map remotely where possible.

For best performance, sharing of data between both GPUs should be limited or done via explicit copies. Alternatively steps such as calling `cudaMemAdviseSetPreferredLocation` should be taken to ensure frequently shared data remains physically resident on CPU or coherent GPU memory as by default accessing software coherent memory requires a fault and migration of data.

On mixed coherency systems, the behavior of `cudaHostRegister` and other host memory access APIs also changes for the software coherent GPU. Instead of using pinned mappings, software coherent GPUs on a mixed coherence system use software mirroring of CPU page tables. This means some GPU accesses may page fault where they would not on a non-mixed coherency system, although such faults are rare and should only occur under memory pressure.

4.1.1.2.2. Direct Unified Memory Access from the Host

Some devices have hardware support for coherent reads, stores and atomic accesses from the host on GPU-resident unified memory. These devices have the attribute

`cudaDevAttrDirectManagedMemAccessFromHost` set to 1. Note that all hardware-coherent

systems have this attribute set for NVLink-connected devices. On these systems, the host has direct access to GPU-resident memory without page faults and data migration. Note that with CUDA managed memory, the `cudaMemAdviseSetAccessedBy` hint with location type `cudaMemLocationTypeHost` is necessary to enable this direct access without page faults, see example below.

System Allocator

Managed

```
__global__ void write(int *ret, int a, int b) {
    ret[threadIdx.x] = a + b + threadIdx.x;
}

__global__ void append(int *ret, int a, int b) {
    ret[threadIdx.x] += a + b + threadIdx.x;
}

void test_malloc() {
    int *ret = (int*)malloc(1000 * sizeof(int));
    // for shared page table systems, the following hint is not necessary
    cudaMemLocation location = {.type = cudaMemLocationTypeHost};
    cudaMemAdvise(ret, 1000 * sizeof(int), cudaMemAdviseSetAccessedBy, location);

    write<<< 1, 1000 >>>(ret, 10, 100);           // pages populated in GPU memory
    cudaDeviceSynchronize();
    for(int i = 0; i < 1000; i++)
        printf("%d: A+B = %d\n", i, ret[i]);      // directManagedMemAccessFromHost=1
                                                // directManagedMemAccessFromHost=0
    append<<< 1, 1000 >>>(ret, 10, 100);           // directManagedMemAccessFromHost=1
    cudaDeviceSynchronize();                     // directManagedMemAccessFromHost=0
    free(ret);
}
```

After `write` kernel is completed, `ret` will be created and initialized in GPU memory. Next, the CPU will access `ret` followed by `append` kernel using the same `ret` memory again. This code will show different behavior depending on the system architecture and support of hardware coherency:

- on devices with `directManagedMemAccessFromHost=1`: CPU accesses to the managed buffer will not trigger any migrations; the data will remain resident in GPU memory and any subsequent GPU kernels can continue to access it directly without inflicting faults or migrations
- on devices with `directManagedMemAccessFromHost=0`: CPU accesses to the managed buffer will page fault and initiate data migration; any GPU kernel trying to access the same data first time will page fault and migrate pages back to GPU memory.

4.1.1.2.3. Host Native Atomics

Some devices, including NVLink-connected devices of hardware-coherent systems, support hardware-accelerated atomic accesses to CPU-resident memory. This implies that atomic accesses to host memory do not have to be emulated with a page fault. For these devices, the attribute `cudaDevAttrHostNativeAtomicSupported` is set to 1.

4.1.1.2.4. Atomic Accesses and Synchronization Primitives

CUDA unified memory supports all atomic operations available to host and device threads, enabling all threads to cooperate by concurrently accessing the same shared memory location. The [libcu++](#) library provides many heterogeneous synchronization primitives tuned for concurrent use between host and device threads, including `cuda::atomic`, `cuda::atomic_ref`, `cuda::barrier`, `cuda::semaphore`, among many others.

On software-coherent systems, atomic accesses from the device to file-backed host memory are not supported. The following example code is valid on hardware-coherent systems but exhibits undefined behavior on other systems:

```

#include <cuda/atomic>

#include <stdio>
#include <fcntl.h>
#include <sys/mman.h>

#define ERR(msg, ...) { fprintf(stderr, msg, ##__VA_ARGS__); return EXIT_FAILURE; }

__global__ void kernel(int* ptr) {
    cuda::atomic_ref{*ptr}.store(2);
}

int main() {
    // this will be closed/deleted by default on exit
    FILE* tmp_file = tmpfile64();
    // need to allocate space in the file, we do this with posix_fallocate here
    int status = posix_fallocate(fileno(tmp_file), 0, 4096);
    if (status != 0) ERR("Failed to allocate space in temp file\n");
    int* ptr = (int*)mmap(NULL, 4096, PROT_READ | PROT_WRITE, MAP_PRIVATE, fileno(tmp_file), 0);
    if (ptr == MAP_FAILED) ERR("Failed to map temp file\n");

    // initialize the value in our file-backed memory
    *ptr = 1;
    printf("Atom value: %d\n", *ptr);

    // device and host thread access ptr concurrently, using cuda::atomic_ref
    kernel<<<1, 1>>>(ptr);
    while (cuda::atomic_ref{*ptr}.load() != 2);
    // this will always be 2
    printf("Atom value: %d\n", *ptr);

    return EXIT_SUCCESS;
}

```

On software-coherent systems, atomic accesses to unified memory may incur page faults which can lead to significant latencies. Note that this is not the case for all GPU atomics to CPU memory on these systems: operations listed by `nvidia-smi -q | grep "Atomic Caps Outbound"` may avoid page faults.

On hardware-coherent systems, atomics between host and device do not require page faults, but may still fault for other reasons that can cause any memory access to fault.

4.1.1.2.5. Memcpy()/Memset() Behavior With Unified Memory

`cudaMemcpy*()` and `cudaMemset*()` accept any unified memory pointer as arguments.

For `cudaMemcpy*()`, the direction specified as `cudaMemcpyKind` is a performance hint, which can have a higher performance impact if any of the arguments is a unified memory pointer.

Thus, it is recommended to follow the following performance advice:

- When the physical location of unified memory is known, use an accurate `cudaMemcpyKind` hint.
- Prefer `cudaMemcpyDefault` over an inaccurate `cudaMemcpyKind` hint.
- Always use populated (initialized) buffers: avoid using these APIs to initialize memory.
- Avoid using `cudaMemcpy*()` if both pointers point to system-allocated memory: launch a kernel or use a CPU memory copy algorithm such as `std::memcpy` instead.

4.1.1.2.6. Overview of Memory Allocators for Unified Memory

For systems with full CUDA unified memory support various different allocators may be used to allocate unified memory. The following table shows an overview of a selection of allocators with their respective features. Note that all information in this section is subject to change in future CUDA versions.

Table 7: Overview of unified memory support of different allocators

API	Placement Policy	Accessible From	Migrate Based On Access [2]	Page Sizes [4] [5]
<code>malloc</code> , <code>new</code> , <code>mmap</code>	First touch/hint [1]	CPU, GPU	Yes [3]	System or huge page size [6]
<code>cudaMallocManaged</code>	First touch/hint	CPU, GPU	Yes	CPU resident: system page size GPU resident: 2MB
<code>cudaMalloc</code>	GPU	GPU	No	GPU page size: 2MB
<code>cudaMallocHost</code> , <code>cudaHostAlloc</code> , <code>cudaHostRegister</code>	CPU	CPU, GPU	No	Mapped by CPU: system page size Mapped by GPU: 2MB

API	Placement Policy	Accessible From	Migrate Based On Access [2]	Page Sizes [4] [5]
Memory pools, location type host: <code>cuMemCreate</code> , <code>cudaMemPoolCreate</code>	CPU	CPU, GPU	No	Mapped by CPU: system page size Mapped by GPU: 2MB
Memory pools, location type device: <code>cuMemCreate</code> , <code>cudaMemPoolCreate</code> , <code>cudaMallocAsync</code>	GPU	GPU	No	2MB

[1] For `mmap`, file-backed memory is placed on the CPU by default, unless specified otherwise through `cudaMemAdviseSetPreferredLocation` (or `mbind`, see bullet points below).

[2] This feature can be overridden with `cudaMemAdvise`. Even if access-based migrations are disabled, if the backing memory space is full, memory might migrate.

[3] File-backed memory will not migrate based on access.

[4] The default system page size is 4KiB or 64KiB on most systems, unless huge page size was explicitly specified (for example, with `mmap` `MAP_HUGETLB` / `MAP_HUGE_SHIFT`). In this case, any huge page size configured on the system is supported.

[5] Page-sizes for GPU-resident memory may evolve in future CUDA versions.

[6] Currently huge page sizes may not be kept when migrating memory to the GPU or placing it through first-touch on the GPU.

The table [Overview of unified memory support of different allocators](#) shows the difference in semantics of several allocators that may be considered to allocate data accessible from multiple processors at a time, including host and device. For additional details about `cudaMemPoolCreate`, see the [Memory Pools](#) section, for additional details about `cuMemCreate`, see the [Virtual Memory Management](#) section.

On hardware-coherent systems where device memory is exposed as a NUMA domain to the system, special allocators such as `numa_alloc_on_node` may be used to pin memory to the given NUMA node, either host or device. This memory is accessible from both host and device and does not migrate. Similarly, `mbind` can be used to pin memory to the given

NUMA node(s), and can cause file-backed memory to be placed on the given NUMA node(s) before it is first accessed.

The following applies to allocators of memory that is shared:

- System allocators such as `mmap` allow sharing the memory between processes using the `MAP_SHARED` flag. This is supported in CUDA and can be used to share memory between different devices connected to the same host. However, this is currently not supported for sharing memory between multiple hosts as well as multiple devices. See [Inter-Process Communication \(IPC\) with Unified Memory](#) for details.
- For access to unified memory or other CUDA memory through a network on multiple hosts, consult the documentation of the communication library used, for example [NCCL](#), [NVSHMEM](#), [OpenMPI](#), [UCX](#), etc.

4.1.1.2.7. Access Counter Migration

On hardware-coherent systems, the access counters feature keeps track of the frequency of access that a GPU makes to memory located on other processors. This is needed to ensure memory pages are moved to the physical memory of the processor that is accessing the pages most frequently. It can guide migrations between CPU and GPU, as well as between peer GPUs, a process called access counter migration.

Starting with CUDA 12.4, access counters are supported system-allocated memory. Note that file-backed memory does not migrate based on access. For system-allocated memory, access counters migration can be switched on by using the `cudaMemAdviseSetAccessedBy` hint to a device with the corresponding device id. If access counters are on, one can use `cudaMemAdviseSetPreferredLocation` set to host to prevent migrations. Per default `cudaMallocManaged` migrates based on a fault-and-migrate mechanism. [\[7\]](#)

The driver may also use access counters for more efficient thrashing mitigation or memory oversubscription scenarios.

[\[7\]](#) Current systems allow the use of access-counter migration with managed memory when the accessed-by device hint is set. This is an implementation detail and should not be relied on for future compatibility.

4.1.1.2.8. Avoid Frequent Writes to GPU-Resident Memory from the CPU

If the host accesses unified memory, cache misses may introduce more traffic than expected between host and device. Many CPU architectures require all memory operations to go through the cache hierarchy, including writes. If system memory is resident on the GPU, this means that frequent writes by the CPU to this memory can cause cache misses, thus transferring the data first from the GPU to CPU before writing the actual value into the requested memory range. On software-coherent systems, this may introduce additional page faults, while on hardware-coherent systems, it may cause higher latencies between CPU operations. Thus, in order to share data produced by the host with the device, consider writing to CPU-resident memory and reading the values directly from the device. The code below shows how to achieve this with unified memory.

System Allocator**Managed**

```
size_t data_size = sizeof(int);
int* data = (int*)malloc(data_size);
// ensure that data stays local to the host and avoid faults
cudaMemLocation location = {.type = cudaMemLocationTypeHost};
cudaMemAdvise(data, data_size, cudaMemAdviseSetPreferredLocation, location)
cudaMemAdvise(data, data_size, cudaMemAdviseSetAccessedBy, location);

// frequent exchanges of small data: if the CPU writes to CPU-resident memo
// and GPU directly accesses that data, we can avoid the CPU caches re-load
// data if it was evicted in between writes
for (int i = 0; i < 10; ++i) {
    *data = 42 + i;
    kernel<<<1, 1>>>(data);
    cudaDeviceSynchronize();
    // CPU cache potentially evicted data here
}
free(data);
```

4.1.1.2.9. Exploiting Asynchronous Access to System Memory

If an application needs to share results from work on the device with the host, there are several possible options:

1. The device writes its result to GPU-resident memory, the result is transferred using `cudaMemcpy*`, and the host reads the transferred data.
2. The device directly writes its result to CPU-resident memory, and the host reads that data.
3. The device writes to GPU-resident memory, and the host directly accesses that data.

If independent work can be scheduled on the device while the result is transferred/accessed by the host, options 1 or 3 are preferred. If the device is starved until the host has accessed the result, option 2 might be preferred. This is because the device can generally write at a higher bandwidth than the host can read, unless many host threads are used to read the data.

1. Explicit Copy

2. Device Direct Write

3. Host Direct Read

```
void exchange_explicit_copy(cudaStream_t stream) {
    int* data, *host_data;
    size_t n_bytes = sizeof(int) * 16;
    // allocate receiving buffer
    host_data = (int*)malloc(n_bytes);
    // allocate, since we touch on the device first, will be GPU-resident
    cudaMallocManaged(&data, n_bytes);
    kernel<<<1, 16, 0, stream>>>(data);
    // launch independent work on the device
    // other_kernel<<<1024, 256, 0, stream>>>(other_data, ...);
    // transfer to host
    cudaMemcpyAsync(host_data, data, n_bytes, cudaMemcpyDeviceToHost, stream);
    // sync stream to ensure data has been transferred
    cudaStreamSynchronize(stream);
    // read transferred data
    printf("Got values %d - %d from GPU\n", host_data[0], host_data[15]);
    cudaFree(data);
    free(host_data);
}
```

Finally, in the Explicit Copy example above, instead of using `cudaMemcpy*` to transfer data, one could use a host or device kernel to perform this transfer explicitly. For contiguous data using the CUDA copy-engines is preferred because operations performed by copy-engines can be overlapped with work on both the host and device. Copy-engines might be used in `cudaMemcpy*` and `cudaMemPrefetchAsync` APIs, but there is no guarantee that copy-engines are used with `cudaMemcpy*` API calls. For the same reason, explicitly copy is preferred over direct host read for large enough data: if both host and device perform work that does not saturate their respective memory systems, the transfer can be performed by the copy-engines concurrently with the work performed by both host and device.

Copy-engines are generally used for both transfers between host and device as well as between peer devices within an NVLink-connected system. Due to the limited total number of copy-engines, some systems may have a lower bandwidth of `cudaMemcpy*` compared to using the device to explicitly perform the transfer. In such a case, if the transfer is in the critical path of the application, it may be preferred to use an explicit device-based transfer.

4.1.2. Unified Memory on Devices with only CUDA Managed Memory Support

For devices with compute capability 6.x or higher but without pageable memory access, see table [Overview of Unified Memory Paradigms](#), CUDA managed memory is fully supported and coherent but the GPU cannot access system-allocated memory. The programming model and performance tuning of unified memory is largely similar to the model as described in the section, [Unified Memory on Devices with Full CUDA Unified Memory Support](#), with the notable exception that system allocators cannot be used to allocate memory. Thus, the following list of sub-sections do not apply:

- [Unified Memory: In-Depth Examples](#)
- [CPU and GPU Page Tables: Hardware Coherency vs. Software Coherency](#)
- [Atomic Accesses and Synchronization Primitives](#)
- [Access Counter Migration](#)
- [Avoid Frequent Writes to GPU-Resident Memory from the CPU](#)
- [Exploiting Asynchronous Access to System Memory](#)

4.1.3. Unified Memory on Windows, WSL, and Tegra

Note

This section is only looking at devices with compute capability lower than 6.0 or Windows platforms, devices with `concurrentManagedAccess` property set to 0.

Devices with compute capability lower than 6.0 or Windows platforms, devices with `concurrentManagedAccess` property set to 0, see [Overview of Unified Memory Paradigms](#), support CUDA managed memory with the following limitations:

- **Data Migration and Coherency:** Fine-grained movement of the managed data to GPU on-demand is not supported. Whenever a GPU kernel is launched all managed memory generally has to be transferred to GPU memory to avoid faulting on memory access. Page faulting is only supported from the CPU side.
- **GPU Memory Oversubscription:** They cannot allocate more managed memory than the physical size of GPU memory.

- **Coherency and Concurrency:** Simultaneous access to managed memory is not possible, because coherence could not be guaranteed if the CPU accessed a unified memory allocation while a GPU kernel is active because of the missing GPU page faulting mechanism.

4.1.3.1. Multi-GPU

On systems with devices of compute capabilities lower than 6.0 or Windows platforms managed allocations are automatically visible to all GPUs in a system via the peer-to-peer capabilities of the GPUs. Managed memory allocations behave similar to unmanaged memory allocated using `cudaMalloc()`: the current active device is the home for the physical allocation but other GPUs in the system will access the memory at reduced bandwidth over the PCIe bus.

On Linux the managed memory is allocated in GPU memory as long as all GPUs that are actively being used by a program have the peer-to-peer support. If at any time the application starts using a GPU that doesn't have peer-to-peer support with any of the other GPUs that have managed allocations on them, then the driver will migrate all managed allocations to system memory. In this case, all GPUs experience PCIe bandwidth restrictions.

On Windows, if peer mappings are not available (for example, between GPUs of different architectures), then the system will automatically fall back to using mapped memory, regardless of whether both GPUs are actually used by a program. If only one GPU is actually going to be used, it is necessary to set the `CUDA_VISIBLE_DEVICES` environment variable before launching the program. This constrains which GPUs are visible and allows managed memory to be allocated in GPU memory.

Alternatively, on Windows users can also set `CUDA_MANAGED_FORCE_DEVICE_ALLOC` to a non-zero value to force the driver to always use device memory for physical storage. When this environment variable is set to a non-zero value, all devices used in that process that support managed memory have to be peer-to-peer compatible with each other. The error `::cudaErrorInvalidDevice` will be returned if a device that supports managed memory is used and it is not peer-to-peer compatible with any of the other managed memory supporting devices that were previously used in that process, even if `::cudaDeviceReset` has been called on those devices. These environment variables are described in [CUDA Environment Variables](#).

4.1.3.2. Coherency and Concurrency

To ensure coherency the unified memory programming model puts constraints on data accesses while both the CPU and GPU are executing concurrently. In effect, the GPU has exclusive access to all managed data and the CPU is not permitted to access it, while any kernel operation is executing, regardless of whether the specific kernel is actively using the data. Concurrent CPU/GPU accesses, even to different managed memory allocations, will cause a segmentation fault because the page is considered inaccessible to the CPU.

For example the following code runs successfully on devices of compute capability 6.x due to the GPU page faulting capability which lifts all restrictions on simultaneous access but fails on pre-6.x architectures and Windows platforms because the GPU program kernel is still active when the CPU touches `y`:

```
__device__ __managed__ int x, y=2;
__global__ void kernel() {
    x = 10;
}
int main() {
    kernel<<< 1, 1 >>>();
    y = 20;           // Error on GPUs not supporting concurrent access

    cudaDeviceSynchronize();
    return 0;
}
```

The program must explicitly synchronize with the GPU before accessing `y` (regardless of whether the GPU kernel actually touches `y` (or any managed data at all)):

```
__device__ __managed__ int x, y=2;
__global__ void kernel() {
    x = 10;
}
int main() {
    kernel<<< 1, 1 >>>();
    cudaDeviceSynchronize();
    y = 20;           // Success on GPUs not supporting concurrent access
    return 0;
}
```

Note that any function call that logically guarantees the GPU completes its work is valid to ensure logically that the GPU work is completed, see [Explicit Synchronization](#).

Note that if memory is dynamically allocated with `cudaMallocManaged()` or `cuMemAllocManaged()` while the GPU is active, the behavior of the memory is unspecified until additional work is launched or the GPU is synchronized. Attempting to access the memory on the CPU during this time may or may not cause a segmentation fault. This does not apply to memory allocated using the flag `cudaMemAttachHost` or `CU_MEM_ATTACH_HOST`.

4.1.3.3. Stream Associated Unified Memory

The CUDA programming model provides streams as a mechanism for programs to indicate dependence and independence among kernel launches. Kernels launched into the same stream are guaranteed to execute consecutively, while kernels launched into different streams are permitted to execute concurrently. See section [CUDA Streams](#).

4.1.3.3.1. Stream Callbacks

It is legal for the CPU to access managed data from within a stream callback, provided no other stream that could potentially be accessing managed data is active on the GPU. In addition, a callback that is not followed by any device work can be used for synchronization: for example, by signaling a condition variable from inside the callback; otherwise, CPU access is valid only for the duration of the callback(s). There are several important points of note:

1. It is always permitted for the CPU to access non-managed mapped memory data while the GPU is active.
2. The GPU is considered active when it is running any kernel, even if that kernel does not make use of managed data. If a kernel might use data, then access is forbidden
3. There are no constraints on concurrent inter-GPU access of managed memory, other than those that apply to multi-GPU access of non-managed memory.
4. There are no constraints on concurrent GPU kernels accessing managed data.

Note how the last point allows for races between GPU kernels, as is currently the case for non-managed GPU memory. In the perspective of the GPU, managed memory functions are identical to non-managed memory. The following code example illustrates these points:

```

int main() {
    cudaStream_t stream1, stream2;
    cudaStreamCreate(&stream1);
    cudaStreamCreate(&stream2);
    int *non_managed, *managed, *also_managed;
    cudaMallocHost(&non_managed, 4);    // Non-managed, CPU-accessible memory
    cudaMallocManaged(&managed, 4);
    cudaMallocManaged(&also_managed, 4);
    // Point 1: CPU can access non-managed data.
    kernel<<< 1, 1, 0, stream1 >>>(managed);
    *non_managed = 1;
    // Point 2: CPU cannot access any managed data while GPU is busy,
    //           unless concurrentManagedAccess = 1
    // Note we have not yet synchronized, so "kernel" is still active.
    *also_managed = 2;    // Will issue segmentation fault
    // Point 3: Concurrent GPU kernels can access the same data.
    kernel<<< 1, 1, 0, stream2 >>>(managed);
    // Point 4: Multi-GPU concurrent access is also permitted.
    cudaSetDevice(1);
    kernel<<< 1, 1 >>>(managed);
    return 0;
}

```

4.1.3.3.2. Managed memory associated to streams allows for finer-grained control

Unified memory builds upon the stream-independence model by allowing a CUDA program to explicitly associate managed allocations with a CUDA stream. In this way, the programmer indicates the use of data by kernels based on whether they are launched into a specified stream or not. This enables opportunities for concurrency based on program-specific data access patterns. The function to control this behavior is:

```

cudaError_t cudaStreamAttachMemAsync(cudaStream_t stream,
                                     void *ptr,
                                     size_t length=0,
                                     unsigned int flags=0);

```

The `cudaStreamAttachMemAsync()` function associates `length` bytes of memory starting from `ptr` with the specified stream. This allows CPU access to that memory region so long as all operations in stream have completed, regardless of whether other streams are active. In effect, this constrains exclusive ownership of the managed memory region by an active GPU to per-stream activity instead of whole-GPU activity. Most importantly, if an allocation is not associated with a specific stream, it is visible to all running kernels regardless of their stream. This is the default visibility for a `cudaMallocManaged()` allocation or a `__managed__` variable; hence, the simple-case rule that the CPU may not touch the data while any kernel is running.

Note

By associating an allocation with a specific stream, the program makes a guarantee that only kernels launched into that stream will touch that data. No error checking is performed by the unified memory system.

Note

In addition to allowing greater concurrency, the use of `cudaStreamAttachMemAsync()` can enable data transfer optimizations within the unified memory system that may affect latencies and other overhead.

The following example shows how to explicitly associate `y` with host accessibility, thus enabling access at all times from the CPU. (Note the absence of `cudaDeviceSynchronize()` after the kernel call.) Accesses to `y` by the GPU running kernel will now produce undefined results.

```
__device__ __managed__ int x, y=2;
__global__ void kernel() {
    x = 10;
}
int main() {
    cudaStream_t stream1;
    cudaStreamCreate(&stream1);
    cudaStreamAttachMemAsync(stream1, &y, 0, cudaMemAttachHost);
    cudaDeviceSynchronize(); // Wait for Host attachment to occur.
    kernel<<< 1, 1, 0, stream1 >>>(); // Note: Launches into stream1.
    y = 20; // Success – a kernel is running but “y”
            // has been associated with no stream.

    return 0;
}
```

4.1.3.3.3. A more elaborate example on multithreaded host programs

The primary use for `cudaStreamAttachMemAsync()` is to enable independent task parallelism using CPU threads. Typically in such a program, a CPU thread creates its own stream for all work that it generates because using CUDA's NULL stream would cause dependencies between threads. The default global visibility of managed data to any GPU stream can make it difficult to avoid interactions between CPU threads in a multi-threaded program. Function `cudaStreamAttachMemAsync()` is therefore used to associate a thread's managed allocations with that thread's own stream, and the association is typically not

changed for the life of the thread. Such a program would simply add a single call to

`cudaStreamAttachMemAsync()` to use unified memory for its data accesses:

```
// This function performs some task, in its own , in its own private stream and
void run_task(int *in, int *out, int length) {
    // Create a stream for us to use.
    cudaStream_t stream;
    cudaStreamCreate(&stream);
    // Allocate some managed data and associate with our stream.
    // Note the use of the host-attach flag to cudaMallocManaged();
    // we then associate the allocation with our stream so that
    // our GPU kernel launches can access it.
    int *data;
    cudaMallocManaged((void **)&data, length, cudaMemAttachHost);
    cudaStreamAttachMemAsync(stream, data);
    cudaStreamSynchronize(stream);
    // Iterate on the data in some way, using both Host & Device.
    for(int i=0; i<N; i++) {
        transform<<< 100, 256, 0, stream >>>(in, data, length);
        cudaStreamSynchronize(stream);
        host_process(data, length);    // CPU uses managed data.
        convert<<< 100, 256, 0, stream >>>(out, data, length);
    }
    cudaStreamSynchronize(stream);
    cudaStreamDestroy(stream);
    cudaFree(data);
}
```

In this example, the allocation-stream association is established just once, and then data is used repeatedly by both the host and device. The result is much simpler code than occurs with explicitly copying data between host and device, although the result is the same.

The function `cudaMallocManaged()` specifies the `cudaMemAttachHost` flag, which creates an allocation that is initially invisible to device-side execution. (The default allocation would be visible to all GPU kernels on all streams.) This ensures that there is no accidental interaction with another thread's execution in the interval between the data allocation and when the data is acquired for a specific stream.

Without this flag, a new allocation would be considered in-use on the GPU if a kernel launched by another thread happens to be running. This might impact the thread's ability to access the newly allocated data from the CPU before it is able to explicitly attach it to a private stream. To enable safe independence between threads, therefore, allocations should be made specifying this flag.

An alternative would be to place a process-wide barrier across all threads after the allocation has been attached to the stream. This would ensure that all threads complete their data/stream associations before any kernels are launched, avoiding the hazard. A

second barrier would be needed before the stream is destroyed because stream destruction causes allocations to revert to their default visibility. The `cudaMemAttachHost` flag exists both to simplify this process, and because it is not always possible to insert global barriers where required.

4.1.3.3.4. Data Movement of Stream Associated Unified Memory

`Memcpy()/Memset()` with stream associated unified memory behaves different on devices where `concurrentManagedAccess` is not set, the following rules apply:

If `cudaMemcpyHostTo*` is specified and the source data is unified memory, then it will be accessed from the host if it is coherently accessible from the host in the copy stream (1); otherwise it will be accessed from the device. Similar rules apply to the destination when `cudaMemcpy*ToHost` is specified and the destination is unified memory.

If `cudaMemcpyDeviceTo*` is specified and the source data is unified memory, then it will be accessed from the device. The source must be coherently accessible from the device in the copy stream (2); otherwise, an error is returned. Similar rules apply to the destination when `cudaMemcpy*ToDevice` is specified and the destination is unified memory.

If `cudaMemcpyDefault` is specified, then unified memory will be accessed from the host either if it cannot be coherently accessed from the device in the copy stream (2) or if the preferred location for the data is `cudaCpuDeviceId` and it can be coherently accessed from the host in the copy stream (1); otherwise, it will be accessed from the device.

When using `cudaMemset*()` with unified memory, the data must be coherently accessible from the device in the stream being used for the `cudaMemset()` operation (2); otherwise, an error is returned.

When data is accessed from the device either by `cudaMemcpy*` or `cudaMemset*`, the stream of operation is considered to be active on the GPU. During this time, any CPU access of data that is associated with that stream or data that has global visibility, will result in a segmentation fault if the GPU has a zero value for the device attribute `concurrentManagedAccess`. The program must synchronize appropriately to ensure the operation has completed before accessing any associated data from the CPU.

1. Coherently accessible from the host in a given stream means that the memory neither has global visibility nor is it associated with the given stream.

2. Coherently accessible from the device in a given stream means that the memory either has global visibility or is associated with the given stream.

4.1.4. Performance Hints

Performance hints allow programmers to provide CUDA with more information about unified memory usage. CUDA uses performance hints to managed memory more efficiently and improve application performance. Performance hints never impact the correctness of an application. Performance hints only affect performance.

Note

Applications should only use unified memory performance hints if they improve performance.

Performance hints may be used on any unified memory allocation, including CUDA managed memory. On systems with full CUDA unified memory support, performance hints can be applied to all system-allocated memory.

4.1.4.1. Data Prefetching

The `cudaMemPrefetchAsync` API is an asynchronous stream-ordered API that may migrate data to reside closer to the specified processor. The data may be accessed while it is being prefetched. The migration does not begin until all prior operations in the stream have completed, and completes before any subsequent operation in the stream.

```
cudaError_t cudaMemPrefetchAsync(const void *devPtr,
                                size_t count,
                                struct cudaMemLocation location,
                                unsigned int flags,
                                cudaStream_t stream=0);
```

A memory region containing `[devPtr, devPtr + count)` may be migrated to the destination device `location.id` if `location.type` is `cudaMemLocationTypeDevice`, or CPU if `location.type` is `cudaMemLocationTypeHost`, when the prefetch task is executed in the given `stream`. For details on `flags`, see the current [CUDA Runtime API documentation](https://docs.nvidia.com/cuda/cuda-runtime-api/).

Consider the simple code example below:

System Allocator

Managed

```

void test_prefetch_sam(const cudaStream_t& s) {
    // initialize data on CPU
    char *data = (char*)malloc(dataSizeBytes);
    init_data(data, dataSizeBytes);
    cudaMemLocation location = {.type = cudaMemLocationTypeDevice, .id = myGpuI

    // encourage data to move to GPU before use
    const unsigned int flags = 0;
    cudaMemPrefetchAsync(data, dataSizeBytes, location, flags, s);

    // use data on GPU
    const unsigned num_blocks = (dataSizeBytes + threadsPerBlock - 1) / threads
mykernel<<<num_blocks, threadsPerBlock, 0, s>>>(data, dataSizeBytes);

    // encourage data to move back to CPU
    location = {.type = cudaMemLocationTypeHost};
    cudaMemPrefetchAsync(data, dataSizeBytes, location, flags, s);

    cudaStreamSynchronize(s);

    // use data on CPU
    use_data(data, dataSizeBytes);
    free(data);
}

```

4.1.4.2. Data Usage Hints

When multiple processors simultaneously access the same data, `cudaMemAdvise` may be used to hint how the data at `[devPtr, devPtr + count)` will be accessed:

```

cudaError_t cudaMemAdvise(const void *devPtr,
                          size_t count,
                          enum cudaMemoryAdvise advice,
                          struct cudaMemLocation location);

```

The example shows how to use `cudaMemAdvise`:

```

init_data(data, dataSizeBytes);
cudaMemLocation location = {.type = cudaMemLocationTypeDevice, .id = myGpuId}

// encourage data to move to GPU before use
const unsigned int flags = 0;
cudaMemPrefetchAsync(data, dataSizeBytes, location, flags, s);

// use data on GPU
const unsigned num_blocks = (dataSizeBytes + threadsPerBlock - 1) / threadsPerBlock;
mykernel<<<num_blocks, threadsPerBlock, 0, s>>>(data, dataSizeBytes);

// encourage data to move back to CPU
location = {.type = cudaMemLocationTypeHost};
cudaMemPrefetchAsync(data, dataSizeBytes, location, flags, s);

cudaStreamSynchronize(s);

// use data on CPU
use_data(data, dataSizeBytes);
cudaFree(data);
}
// test-prefetch-managed-end

static const int maxDevices = 1;
static const int maxOuterLoopIter = 3;
static const int maxInnerLoopIter = 4;

// test-advise-managed-begin
void test_advise_managed(cudaStream_t stream) {
    char *dataPtr;
    size_t dataSize = 64 * threadsPerBlock; // 16 KiB

```

Where `advice` may take the following values:

- **`cudaMemAdviseSetReadMostly`:**

This implies that the data is mostly going to be read from and only occasionally written to. In general, it allows trading off read bandwidth for write bandwidth on this region.

- **`cudaMemAdviseSetPreferredLocation`:**

This hint sets the preferred location for the data to be the specified device's physical memory. This hint encourages the system to keep the data at the preferred location, but does not guarantee it. Passing in a value of `cudaMemLocationTypeHost` for `location.type` sets the preferred location as CPU memory. Other hints, like `cudaMemPrefetchAsync`, may override this hint and allow the memory to migrate away from its preferred location.

- **`cudaMemAdviseSetAccessedBy`:**

In some systems, it may be beneficial for performance to establish a mapping into memory before accessing the data from a given processor. This hint tells the system that the data will be frequently accessed by `location.id` when `location.type` is `cudaMemLocationTypeDevice`, enabling the system to assume that creating these mappings pays off. This hint does not imply where the data should reside, but it can be combined with `cudaMemAdviseSetPreferredLocation` to specify that. On hardware-coherent systems, this hint switches on access counter migration, see [Access Counter Migration](#).

Each advice can be also unset by using one of the following values:

`cudaMemAdviseUnsetReadMostly`, `cudaMemAdviseUnsetPreferredLocation` and `cudaMemAdviseUnsetAccessedBy`.

The example shows how to use `cudaMemAdvise`:

System Allocator

Managed

```

void test_advise_sam(cudaStream_t stream) {
    char *dataPtr;
    size_t dataSize = 64 * threadsPerBlock; // 16 KiB

    // Allocate memory using malloc or cudaMallocManaged
    dataPtr = (char*)malloc(dataSize);

    // Set the advice on the memory region
    cudaMemLocation loc = {.type = cudaMemLocationTypeDevice, .id = myGpuId};
    cudaMemAdvise(dataPtr, dataSize, cudaMemAdviseSetReadMostly, loc);

    int outerLoopIter = 0;
    while (outerLoopIter < maxOuterLoopIter) {
        // The data is written by the CPU each outer loop iteration
        init_data(dataPtr, dataSize);

        // The data is made available to all GPUs by prefetching.
        // Prefetching here causes read duplication of data instead
        // of data migration
        cudaMemLocation location;
        location.type = cudaMemLocationTypeDevice;
        for (int device = 0; device < maxDevices; device++) {
            location.id = device;
            const unsigned int flags = 0;
            cudaMemPrefetchAsync(dataPtr, dataSize, location, flags, stream);
        }

        // The kernel only reads this data in the inner loop
        int innerLoopIter = 0;
        while (innerLoopIter < maxInnerLoopIter) {
            mykernel<<<32, threadsPerBlock, 0, stream>>>((const char *)dataPtr, dat
            innerLoopIter++;
        }
        outerLoopIter++;
    }

    free(dataPtr);
}

```

4.1.4.3. Memory Discarding

The `cudaMemDiscardBatchAsync` API allows applications to inform the CUDA runtime that the contents of specified memory ranges are no longer useful. The Unified Memory driver performs automatic memory transfers due to fault-based migration or memory evictions to support device memory oversubscription. These automatic memory transfers can sometimes be redundant, which severely decreases performance. Marking an address range as 'discard' will inform the Unified Memory driver that the application has consumed the contents in the range and there is no need to migrate this data on prefetches or page evictions in order to make room for other allocations. Reading a discarded page without a

subsequent write access or prefetch will yield an indeterminate value. Whereas any new writes after the discard operation is guaranteed to be seen by a subsequent read access. Concurrent accesses or prefetches to address ranges being discarded will result in undefined behavior.

```
cudaError_t cudaMemDiscardBatchAsync(void **dptrs,
                                     size_t *sizes,
                                     size_t count,
                                     unsigned long long flags,
                                     cudaStream_t stream);
```

The function performs a batch of memory discards on address ranges specified in `dptrs` and `sizes` arrays. Both arrays must be of the same length as specified by `count`. Each memory range must refer to managed memory allocated via `cudaMallocManaged` or declared via `__managed__` variables.

The `cudaMemDiscardAndPrefetchBatchAsync` API combines both discard and prefetch operations. Calling `cudaMemDiscardAndPrefetchBatchAsync` is semantically equivalent to calling `cudaMemDiscardBatchAsync` followed by `cudaMemPrefetchBatchAsync`, but is more optimal. This is useful when the application needs the memory to be on the target location but does not need the contents of the memory.

```
cudaError_t cudaMemDiscardAndPrefetchBatchAsync(void **dptrs,
                                                size_t *sizes,
                                                size_t count,
                                                struct cudaMemLocation *prefetchLocs,
                                                size_t *prefetchLocIdxs,
                                                size_t numPrefetchLocs,
                                                unsigned long long flags,
                                                cudaStream_t stream);
```

The `prefetchLocs` array specifies the destinations for prefetching, while `prefetchLocIdxs` indicates which operations each prefetch location applies to. For example, if a batch has 10 operations and the first 6 should be prefetched to one location while the remaining 4 to another, then `numPrefetchLocs` would be 2, `prefetchLocIdxs` would be {0, 6}, and `prefetchLocs` would contain the two destination locations.

Important considerations:

- Reading from a discarded range without a subsequent write or prefetch will return an

[Privacy Policy](#) | [Your Privacy Choices](#) | [Terms of Service](#) | [Accessibility](#) | [Corporate Policies](#) | [Product Security](#) | [Contact](#)

Copyright © 2007-2026, NVIDIA Corporation & affiliates. All rights reserved.

Last updated on May 27, 2026.